

Compiladores, 2025/2026

Trabalho prático, parte 2

– Fava: variáveis globais, instruções de controlo de fluxo –

Fernando Lobo

1 Introdução

Neste trabalho vamos estender a linguagem Fava para permitir declarações de variáveis globais, instrução de afectação, e instruções de controlo de fluxo. Especificamente, para além da instrução *print* existente no trabalho 1, passaremos a ter as seguintes instruções:

- afectação
- bloco
- while
- if / if-else
- vazia

2 Alterações à linguagem

2.1 Programa

Um programa em Fava sintacticamente válido passa agora a ser constituído por uma sequência de zero ou mais declarações de variáveis, seguido de uma sequência de uma ou mais instruções.

2.2 Declaração de variáveis

A declaração de variáveis é feita de modo análogo ao C e Java. Isto é, indicamos o tipo de dados, seguido de lista de nomes de variáveis separados por vírgula, e

terminando com um ponto-e-vírgula. O tipo de dados pode ser um dos 4 tipos suportados pela linguagem Fava (`bool`, `integer`, `real`, `string`), que passam a ser palavras reservadas¹.

Exemplo:

```
integer x, y, soma;  
bool b;
```

Opcionalmente, uma variável ao ser declarada pode ser inicializada com o valor de uma expressão, tal como em C e Java. Para tal usa-se o sinal de afectação `:=` seguido da expressão.

Exemplo:

```
integer x := 1, y, soma := 0;  
bool b := false;
```

O nome de uma variável obedece à mesma regra que é usada na linguagem C. Isto é, deve começar por uma letra ou por um underscore, e depois pode vir uma sequência de letras, dígitos, ou underscores. Note, porém, que não há distinção entre minúsculas e maiúsculas em Fava, pelo que `soma` e `SoMa` designam o mesmo.

Na fase de análise semântica deve ser feito a verificação de tipos. Para além disso, deve ser reportado um erro se fizermos referência a uma variável não declarada, ou se tentarmos declarar uma variável que já foi declarada anteriormente.

Uma variável que não é inicializada fica armazenada na memória da máquina virtual com o valor `NULL`, e o acesso a uma variável nessas condições deve gerar um erro de runtime pela máquina virtual.

2.3 Novo tipo de expressão

Uma vez que é permitido variáveis, também é obviamente permitido ter variáveis no contexto de expressões. Por exemplo, `n + 1` passa a ser uma expressão sintacticamente válida na linguagem Fava. Na fase de verificação de tipos (*type checking*) deve usar as regras especificadas no trabalho anterior, tendo presente que o tipo de uma variável é obtido através da sua declaração.

¹Palavras reservadas não podem ser usadas como nomes de variáveis, funções, etc.

2.4 Novos tipos de instrução

Afectação

A instrução de afectação permite atribuir um valor a uma variável. A sintaxe é parecida à da linguagem C ou Java, mas usa-se o sinal `:=` como sinal da afectação, em vez do sinal de `=` usado em C e Java. A sintaxe é: nome de variável, seguido do sinal de afectação `:=` seguido de uma expressão, seguido de um ponto-e-vírgula. Exemplo:

```
x := 4;
```

Bloco

Um bloco, usualmente também designado por instrução composta, é uma instrução que serve para agrupar instruções. Usamos chavetas, tal como em C e Java. O início do bloco é indicado por uma chaveta a abrir e o fim do bloco é indicado por uma chaveta a fechar. O conteúdo do bloco é uma sequência de zero ou mais instruções. Exemplo:

```
{
    print x;
    x := x+1;
}
```

While

A instrução *while* é análoga ao ciclo while em C e Java. A instrução deve começar com a palavra reservada **while**, seguido de um parêntesis curvo a abrir, seguido de uma expressão, seguido de um parêntesis curvo a fechar, seguido de uma instrução. Exemplo:

```
while (x < 10) {
    print x;
    x := x+1;
}
```

Na fase de análise semântica deve ser garantido que a expressão de controlo do ciclo tem de ser do tipo bool.

If / If-Else

Esta instrução é análoga à instrução *if / if-else* do C e Java. A instrução começa com a palavra reservada **if**, seguido de um parêntesis curvo a abrir, seguido de uma expressão, seguido de um parêntesis curvo a fechar seguido de uma instrução, e opcionalmente poderá ter a parte *else*, em que aparece a palavra reservada **else** seguido de uma instrução. Exemplo:

```
if (x < 10)
    print "ola";
else {
    print x;
    x := x+1;
}
```

Na fase de análise semântica deve ser garantido que a expressão do *if* tem de ser do tipo bool.

Vazia

Um ponto-e-vírgula é uma instrução. (É uma instrução que não faz nada.)

3 Máquina Virtual F

O vosso compilador de Fava deverá gerar bytecodes para a máquina virtual F (versão 2). Esta nova versão da máquina virtual é uma extensão daquela que fizeram no 1º trabalho. Para além da *constant pool* e do *runtime stack*, vai também incluir uma *memória* para armazenar as variáveis globais, bem como mais algumas instruções necessárias para alocar posições de memória, fazer leitura e escrita da memória, e ainda instruções que efectuem saltos condicionais e incondicionais.

A especificação desta nova versão da máquina virtual F está disponível num ficheiro à parte na tutoria: **FVM-mini-2.pdf**

4 Reporte de erros

À semelhança do 1º trabalho, no caso de haver erros lexicais ou de parsing, o programa deve limitar-se a dizer **Input has lexical errors** ou **Input has parsing errors**, consoante o caso, e terminar a execução.

Também à semelhança do que aconteceu no 1º trabalho, devem reportar os eventuais erros semânticos que possam haver. Caso o input tenha erros semânticos, devem

reportar uma mensagem de erro indicando a linha onde o erro ocorre, seguido de mensagem apropriada. Note que agora para além dos erros de consistência de tipos, pode também haver erros de identificação de nomes tais como variáveis não declaradas, ou tentativa de declarar algo que já foi declarado. (ver exemplos A, B, C, no apêndice).

5 Requisitos

- O trabalho deve ser feito em Java usando o ANTLR 4, e submetido ao mooshak.
- Devem submeter um ficheiro zip que deverá conter uma pasta chamada `src` onde está todo o código que desenvolveram.
- A gramática deve chamar-se `Fava.g4` e deverá estar na pasta `src`
- A pasta `src` deverá conter o código gerado pelo ANTLR, supostamente numa pasta/package chamada `Fava`
- Nota importante: NÃO DEVEM incluir o ficheiro `antlr-4.13.2-complete.jar` na vossa zip. O servidor do mooshak já lá tem esse ficheiro.
- No código apenas pode haver um ficheiro java que tenha o método `main`. Não pode haver mais nenhum `main`, mesmo que esteja comentado. O nome do ficheiro que tem o método `main` deve chamar-se `FavaCompileAndRun.java`
- O vosso código é submetido ao mooshak, pelo que deverá poder ler o input a partir do *standard input*. Não obstante, o código deve estar também preparado para poder receber o input (nome do ficheiro `fava` a compilar, e eventuais flags) a partir de argumentos passados à função `main`.
- O método `main` deverá definir as seguintes 2 flags que servem para controlar o modo como a emissão de erros é feita.

```
boolean showLexerErrors = false;
boolean showParserErrors = false;
```

Quando o código é submetido ao mooshak, as flags devem ter o valor `false` de modo a não mostrar as mensagens de erro. Porém, se o valor das flags for alterado para `true`, o vosso programa é suposto emitir os respectivos erros com mensagens apropriadas.

- Para efeitos de submissão ao mooshak:
 - se o input tiver erros lexicais, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input has lexical errors`
 - se não tiver erros lexicais, mas tiver erros de parsing, o programa deverá enviar a seguinte mensagem para o output e terminar: `Input has parsing errors`

- se não tiver erros lexicais nem de parsing, mas tiver erros semânticos, o programa deverá enviar mensagens de erro apropriadas para o output e terminar (ver exemplos).
- se não tiver erros lexicais, nem de parsing, nem semânticos, o programa deverá gerar bytecodes de acordo com a especificação da máquina virtual e guardá-los num ficheiro com o nome `bytecodes.bc`. Após ser gerado, esse ficheiro deverá ser lido de imediato e a máquina virtual deverá executar os bytecodes.

6 Sobre o Mooshak

- <http://deei-mooshak.ualg.pt/~flobo/>, concurso Comp26, problema B.
- Não use caracteres acentuados no código, nem mesmo nos comentários, uma vez que o mooshak poderá dar erros por causa disso.
- O compilador de Java instalado no Mooshak é o openjdk versão 21. Se no vosso desenvolvimento usarem um JDK mais recente, recomendo que configurem o vosso IDE para não usar funcionalidades do Java posteriores à versão que está no Mooshak. (Se usarem o IntelliJ vão a 'Project Structure' → 'Language Level', e escolham 21.

7 Validação e avaliação

Os trabalhos apenas serão validados e avaliados após a discussão individual a ter lugar nas 2 últimas semanas de aulas do semestre. Faz-se notar que até ao final do semestre haverá mais um trabalho para além deste.

Não basta terem accepted no mooshak para automaticamente terem boa nota. Aliás, é possível terem boa nota mesmo que não tenham accepted no mooshak.

Os trabalhos serão avaliados de acordo com a clareza e qualidade do código implementado, pela correcta implementação do compilador e máquina virtual, pelo cumprimento dos requisitos acima enunciados, e pelo desempenho individual durante a validação/discussão.

8 Prazo de entrega

O trabalho deve ser realizado em grupo, de acordo com as inscrições em grupo escolhida na tutoria. Deverão submeter o vosso código ao mooshak até às 23:59 do dia 26/Abr/2026. (**NOTA:** Não serão aceites entregas fora de prazo, nem que seja por um só minuto. Não devem deixar a submissão para os últimos instantes. Podem

submeter o vosso trabalho várias vezes. Apenas será considerada a última submissão que tiver 'Accepted', ou no caso de não terem Accepted contará a última submissão.

Apêndice A Exemplos de inputs/outputs para o mooshak

Exemplo A

Input

```
1  bool b, p;  
2  integer x := 3.14, y, soma;  
3  real r := "ualg";  
4  string s;  
5  b := 99;  
6  r := 3;  
7  s := 5;  
8  x := 33;  
9  x := 3.14;  
10 print x;
```

Output

```
1  error in line 2: operator := is invalid between integer and real  
2  error in line 3: operator := is invalid between real and string  
3  error in line 5: operator := is invalid between bool and integer  
4  error in line 7: operator := is invalid between string and integer  
5  error in line 9: operator := is invalid between integer and real
```


Exemplo B

Input

```
1   integer x,y,soma;  
2   bool b,x;  
3   integer SOMA;  
4   print 5;  
5   print 8 + xpto;
```

Output

```
error in line 2: x already declared  
error in line 3: SOMA already declared  
error in line 5: xpto not declared
```

Exemplo C

Input

```
1  integer i := 1;  
2  while (i) {  
3      print i;  
4      if ("esta a chover")  
5          print "abre o guarda-chuva";  
6      i := i + 1;  
7  }  
8  print "tchau";
```

Output

```
error in line 2: while expression must be of type bool  
error in line 4: if expression must be of type bool
```

Exemplo D

Input

```
print "ola";;  
;;;;  
print 1+2;
```

Output

```
*** Constant pool ***  
0: "ola"  
*** Instructions ***  
0: sconst 0  
1: sprint  
2: iconst 1  
3: iconst 2  
4: iadd  
5: iprint  
6: halt  
*** VM output ***  
ola  
3
```

Exemplo E

Input

```
integer x;  
x := 6;  
if (x mod 2 = 0)  
    print x || " eh par";  
else  
    print x || " eh impar";
```

Output

```
*** Constant pool ***  
0: " eh par"  
1: " eh impar"  
*** Instructions ***  
0: galloc 1  
1: iconst 6  
2: gstore 0  
3: gload 0  
4: iconst 2  
5: imod  
6: iconst 0  
7: ieq  
8: jumpf 15  
9: gload 0  
10: itos  
11: sconst 0  
12: sconcat  
13: sprint  
14: jump 20  
15: gload 0  
16: itos  
17: sconst 1  
18: sconcat  
19: sprint  
20: halt  
*** VM output ***  
6 eh par
```

Exemplo F

Input

```
Integer i := 1;
WHILE (i <= 5) {
    print I;
    i := i + 1;
}
print "tchau";
```

Output

```
*** Constant pool ***
0: "tchau"
*** Instructions ***
0: galloc 1
1: iconst 1
2: gstore 0
3: gload 0
4: iconst 5
5: ileq
6: jumpf 14
7: gload 0
8: iprint
9: gload 0
10: iconst 1
11: iadd
12: gstore 0
13: jump 3
14: sconst 0
15: sprint
16: halt
*** VM output ***
1
2
3
4
5
tchau
```

Exemplo G

Input

```
real r := 88;
integer i := 5, j := 2, k;
bool b := true;
real x;

print b;
x := 2;
k := 5;
print x;
print i + j + k;
print r;
```

Output

```
*** Constant pool ***
*** Instructions ***
0: galloc 1
1: iconst 88
2: itod
3: gstore 0
4: galloc 3
5: iconst 5
6: gstore 1
7: iconst 2
8: gstore 2
9: galloc 1
10: tconst
11: gstore 4
12: galloc 1
13: gload 4
14: bprint
15: iconst 2
16: itod
17: gstore 5
18: iconst 5
19: gstore 3
20: gload 5
21: dprint
22: gload 1
23: gload 2
```

```
24: iadd
25: gload 3
26: iadd
27: iprint
28: gload 0
29: dprint
30: halt
*** VM output ***
true
2.0
12
88.0
```

Exemplo H

Input

```
integer i,j,k;  
bool b;  
  
b := true;  
print b;  
i := 5;  
print i + j;
```

Output

```
*** Constant pool ***  
*** Instructions ***  
0: galloc 3  
1: galloc 1  
2: tconst  
3: gstore 3  
4: gload 3  
5: bprint  
6: iconst 5  
7: gstore 0  
8: gload 0  
9: gload 1  
10: iadd  
11: iprint  
12: halt  
*** VM output ***  
true  
runtime error: accessing a NULL value
```